CS673 · FINAL ITERATION

SAFEDROP

Inventory Management & Checkout Service

Team 3 · Final Review and Code Walkthrough · October 2026

Alex Picard	Alexa Stein	Mateus Silva	Orelmis Toribio	Amber Rastella
Configuration Lead	Team Lead	Requirements Lead	QA Lead	Security Lead
The platform	*Members & groups*	*Catalogue & units*	*Request lifecycle*	*Audit & AI search*

Each of us presents our own code, in the order a request travels through it · Live demo on staging

github.com/BUMETCS673/CS673OLF26P3 · safe-drop-five.vercel.app

Know Where Your Equipment Is: Finished

Alex

The question we set out to answer

Who has unit #247 right now, who approved it, and did it come back?

- Iteration 1 built the loop: browse, request, approve, check out, return, all audited.
- Iteration 2 made it configurable per organization: who approves, who may borrow, who confirms a return.
- Iteration 3 redesigned the interface, made AI search dependable and measurable, and closed the review gaps.

One structure, growing

	Iteration 1	Iteration 2	Final
Screens	16	19	**20**
API routes	29	48	**53**
Collections	7	8	**9**
Request state transitions	10	15	**15**
Audit actions	13	25	**25**
Automated tests passing	921	1,508	**1,645**

The ninth collection is SearchQueryLog: AI search telemetry with no query text and no user ids.

Iteration 1 and 2 figures from those reviews. Final column counted from main at cce52f5 (11 Oct);

One Request, Five Builders

Alex

ALEX

Arrives safely

Session from the token; route table authorizes and validates before any feature code runs.

ALEXA

May I borrow it?

Member must be active and in an allowed group, checked on submit and again on approval.

MATEUS

Is a unit free?

The catalogue, its units, maintenance, and similar items when nothing is free.

ORELMIS

Who decides, what next?

Approval policy, the transition table, two-step return, expiry and overdue.

AMBER

Recorded and findable

Audit log and chain of custody read every step; AI search finds the item.

You can see it in one call stack

A request from Mateus's asset page passes Alex's route guard into Orelmis's submit(), which asks Alexa's isEligible() and commits its audit entry in the same transaction, the entry Amber's audit log and chain of custody read back.

POST /api/requests → routes/define.js → checkout.service.js submit() → group.service.js isEligible() → audit entry. Presented in this order.

Alex: The Platform Everyone Builds On

Alex

What I built

ITER 1 — Project skeleton: layered backend, middleware chain, models, Docker Compose; sign-in and forgot password (SCRUM-21, 22); admin dashboard (SCRUM-102)

ITER 1 — CI pipeline, CodeQL, staging deploy, OWASP ZAP (SCRUM-126), daily backup (SCRUM-127)

ITER 2 — Restricted equipment (SCRUM-150); due-date badges (SCRUM-143, Lab 3); ZAP on every release

ITER 3 — Full UI redesign: side rail, Ctrl+K palette, request board; AI-usage headers (SCRUM-216)

Every pull request

1. **Push / PR** — protected main
2. **Six checks** — lint · format · tests (≥80%) · audit · Docker · CodeQL
3. **Review** — a second member approves
4. **Merge → staging** — Vercel + Render redeploy
5. **Release → ZAP** — dynamic scan, report attached

Workflows: ci.yml, codeql.yml, zap.yml, backup.yml. Staging: safe-drop-five.vercel.app and safedrop-ony6.onrender.com.

Code: No Route Without a Permission

Alex

code/backend/src/routes/define.js · defineRoute(), lines 83–124 (trimmed)

```
export function defineRoute(router, spec, handler) {
  const { method, path, permission, public: isPublic =  false,
          schemas } = spec ?? {};
  // ...method and handler checks...
  if (TENANT_PARAM.test(path)) {
    throw new Error(`${label} carries a tenant id in the URL;
      orgId comes only from the token (SR-2)`);
  }
  if (path.includes(':') && !schemas?.params) {
    throw new Error(`${label} must declare schemas.params` );
  }
  if (!isPublic && !permission) {
    throw new Error(`${label} must declare a permission
      (SDD §6.4 deny-by-default) or public:  true`);
  }
  const chain = [
    isPublic ? markPublic() : authorize(permission),
    validate(schemas ?? {}),
    guarded(handler),
  ];
  router[verb](path, ...chain);
}
```

1
2
3
4

What it enforces

1. **No tenant id in a URL**
   A route like /orgs/:orgId cannot even be registered. The server refuses to start.

2. **Every parameter validated**
   A path parameter without a schema is a startup error.

3. **Deny by default**
   No permission and no public flag means no route, caught at boot, not in production.

4. **One chain for all 53 routes**
   Authorize, validate, then the controller, the same for every feature.

assertRoutesDeclarePermission() re-checks the finished app; routePermissions.test.js walks every route against the permission matrix.

LIVE ON STAGING · ~1:45

Alex

Alex: The Platform, Running

1

Landing page

safe-drop-five.vercel.app, the redesigned front door

2

Sign in to the demo org

Organization slug, email, password: org is part of the credential

3

Tour the layout

Primary and Manage rails; members see less, the API refuses the rest

4

Command palette

Ctrl+K → "aud" → Enter; back to Overview

5

The pipeline

GitHub Actions: latest run on main, all green

Alexa: Who Belongs, and Who May Borrow

Amber

What I built

- **ITER 1** — Member lifecycle: list members, invite with an initial password, change role; the Users page
- **ITER 1** — SDD v0.2 change set: the open design decisions, numbered so none was settled silently
- **LAB 3** — Member deactivation, test-first: a deactivated account cannot sign in or borrow (SCRUM-142)
- **ITER 2** — User groups end to end: model, repository, service, 7 routes, 5 audit actions, 460-line integration suite (SCRUM-149)

How it connects

isEligible() is the single answer to "may this person borrow this?". Orelmis's submit() and approve() call it, Mateus's similar items filter by the same rule, and Alex's Restricted badge displays it.

Key files

```
services/group.service.js
models/UserGroup.js
repositories/group.repository.js
services/auth.service.js
```

Team lead throughout: sprint planning, the board, and the release checklist.

Code: Eligible Means In a Group and Active

Amber

code/backend/src/services/group.service.js · isEligible(), line 474

```
export async function isEligible(orgId, userId, asset, opts = {}) {
  const groupIds = (asset.allowedGroupIds ?? []).map(String);
  if (groupIds.length === 0) {
    return true;
  }
  const inAnyGroup =
    await groupRepo.isMemberOfAny(orgId, groupIds, userId, opts);
  if (!inAnyGroup) {
    return false;
  }
  const user = await userRepo.findById(orgId, userId, opts);
  return Boolean(user) && !user.deactivatedAt;
}
```

code/backend/src/services/auth.service.js · login(), deactivation check

```
const ok = await verifyPassword(password, hashToCompare);
// Checked after the bcrypt comparison above, never before it.
const deactivated = Boolean(user?.deactivatedAt);
if (!user || !ok || deactivated) {
  throw new AuthError(INVALID_CREDENTIALS);
}
```

What it decides

1. **Unrestricted is open**
   No allowed groups means anyone in the organization may borrow.
2. **In at least one group**
   Scoped by orgId, like every repository call.
3. **And still active**
   Deactivated members stay listed for admins but are eligible for nothing.
4. **Same answer, same time**
   Deactivation is checked after bcrypt, so it is rejected as slowly as a wrong password, with the same message.

Called from checkout.service.js submit() and approve(), and asset.service.js get(); filterEligible() applies the same rule to similar items.

LIVE ON STAGING · ~1:30

Amber

Alexa: Members and Groups, Running

1

Invite a member

Users → Invite "Dana Demo" as a Member, with an initial password

2

Create a group

Groups → "Certified Drone Pilots" → add Dana

3

See it as the member

As Dana: the drone shows Restricted and who can borrow it

4

Deactivation

deactivate.test.js passing: same 401 as a wrong password

Mateus: The Catalogue and Its Units

Mateus

What I built

- ITER 1 — Catalogue list and detail (SCRUM-115); asset create, edit, retire UI (SCRUM-122) and write endpoints (SCRUM-134)
- ITER 1 — The request form on the asset page (SCRUM-124)
- LAB 3 — Unit maintenance: in and out of service with a compare-and-set, audited in the same transaction (SCRUM-141)
- ITER 2 — "Similar items available now": calls the model only when every unit is out (SCRUM-151)
- ITER 3 — Refactoring PR: Extract Function across the checkout handlers (SCRUM-225, PR #75)

How it connects

Similar items runs on Amber's Foundry pipeline and filters with Alexa's eligibility rule. The request form starts every request Orelmis's lifecycle handles, and maintenance shows up in Alex's dashboard.

Key files

```
services/asset.service.js
services/ai/assetSearch.service.js
components/SimilarItems.jsx
components/RequestUnitForm.jsx
```

Maintenance only succeeds from AVAILABLE, so a unit that is reserved or out can never be pulled into the repair shop by a race.

Code: Our Refactoring PR, Before and After

Mateus

BEFORE

approve(), and again in deny()

```
const request =
  await checkoutRequestRepo.findById(orgId, requestId);
if (!request) {
  throw new NotFoundError( 'Request not found' );
}
const org = await organizationRepo.findById(orgId);
const decision =
  policyFor(org).canDecide(request, actor);
if (!decision.allowed) {
  throw new ForbiddenError(decision.reason);
}
// ...then a compare-and-set and its
// "if (!updated) throw" in every handler
```

AFTER

approve(), deny() and the other handlers

```
const request = await loadRequest(orgId, requestId);
await assertMayDecide(orgId, request, actor);

const updated = await commitTransition(
  orgId, requestId, {
    from: request.state,
    to: S.APPROVED,
    expectedState: S.PENDING,
    patch: { decidedBy: actor.userId },
  }, { session });
// one definition each of: 404-not-403,
// separation of duties, the race
```

Extract Function, three times. loadRequest / loadOwnRequest, assertMayDecide and commitTransition now each exist once. 158 lines in, 85 out of checkout.service.js, no test edited, every test still passing: no behavior change.

SCRUM-225, PR #75, merged 10 Oct. The "after" patch is abbreviated; see checkout.service.js approve().

LIVE ON STAGING · ~1:30

Mateus

Mateus: The Catalogue, Running

1

Create an asset

Catalog → New asset "Zoom H5 recorder", category Audio

2

Add a unit

Add a unit with tag "h5-final-01"

3

Maintenance

Unit → maintenance; Overview tile ticks up; bring it back

4

Similar items

As member, on an asset with every unit out: similar items

Orelmis: Everything After "Request"

Orelmis

What I built

ITER 1 — Request backend: create, cancel, approve, deny, checkout, list; approval queue UI; the seed data everyone tests against

LAB 3 — Overdue loans: markOverdue and the admin endpoint (SCRUM-140)

ITER 2 — Configurable approvals: org default plus per-asset override (SCRUM-148); two-step return and pickup expiry (SCRUM-205)

ITER 3 — Request dates read in the user's time zone (SCRUM-240); search telemetry tests (SCRUM-206)

How it connects

submit() is where the team's code meets: it reserves Mateus's unit, asks Alexa's isEligible(), asks the approval policy, and commits Amber's audit entry in one transaction. Alex's request board drives the same transitions.

Key files

```
services/checkout.service.js
services/policies/approvalPolicy.js
checkoutRequest.repository.js
pages/RequestDetailPage.jsx
```

Every transition goes through one table and commits with its audit entry; SCRUM-240 closed the MT-21 partial from the last review.

Code: Rules That Fail Closed

Orelmis

code/backend/src/services/policies/approvalPolicy.js · effectiveApprovalMode()

```
export function effectiveApprovalMode(asset, org) {
  const assetMode = asset?.approvalMode;
  if (assetMode !== APPROVAL_MODE.INHERIT
      && ORG_APPROVAL_MODE_LIST.includes(assetMode)) {
    return assetMode;
  }
  if (assetMode !== undefined && assetMode !==  null
      && assetMode !== APPROVAL_MODE.INHERIT) {
    return APPROVAL_MODE.REQUIRED;
  }
  const orgMode = org?.approvalSettings?.defaultMode;
  return ORG_APPROVAL_MODE_LIST.includes(orgMode)
```

1 2 3

code/backend/src/repositories/checkoutRequest.repository.js · lateFilter() (reflowed)

```
function lateFilter(orgId, asOf, states) {
  return { orgId, state: mongoose.trusted({ $in: states }),
           dueAt: mongoose.trusted({ $lt: asOf }) };
}
```

4

Why it is safe

1. **The asset decides first**
   Required or auto on the asset wins.
2. **Unknown means required**
   A value that is present but not valid never auto-approves.
3. **Then the org default**
   And if that is missing or invalid, required again.
4. **One definition of "late"**
   The dashboard count and markOverdue both build on it, so they cannot disagree.

Both feed TRANSITIONS in checkout.service.js, which grew from 10 to 15 moves: return pending, expired and overdue among them.

LIVE ON STAGING · ~1:45

Orelmis

Orelmis: A Request, End to End

1

Approval settings

Settings: Required vs Approve automatically; pickup window

2

Auto-approved

Member requests an auto asset: approved, unit held

3

Needs an approver

Required asset: approver approves and records handoff

4

Two-step return

Member starts return with a condition; approver confirms

5

Overdue

Overview → Overdue tile → View overdue

Quality Assurance: Where Testing Finished

Orelmis

1,645	95.7%	96.3%	0
automated tests passing	backend line coverage	frontend line coverage	npm audit findings
1,508 at Iteration 2	gate fails below 80%	gate fails below 80%	at high or above

Three suites carry the security design

routePermissions	every route against the permission matrix, and 401 without a session
tenantIsolation	another organization's ids return 404, not 403
checkout.transitions	every legal and illegal move of the state machine

One Outstanding Manual test

- MT-21 (dates shifting between time zones): FIXED by SCRUM-240.
- MT-19 (Available count after maintenance): open defect
- Test-first throughout: each acceptance criterion committed red before the code that passes it.

Integration tests run against a real MongoDB replica set in memory.

Amber: The Record, and Finding Things

Amber

What I built

ITER 1 — Audit log API and page with filters (SCRUM-46, 51, 52); chain of custody per asset (SCRUM-29)

ITER 1 — Rate limit on public organization creation (SCRUM-114); insert-only audit DB role, written and tested (SCRUM-125)

ITER 2 — AI-assisted search: Foundry client, candidate cap, id filter, plain-search fallback (SCRUM-103, 145, 146, 200, 201)

ITER 3 — Retry with Retry-After, word-match fallback (SCRUM-243, 244); telemetry without query text (SCRUM-206); AI search page (SCRUM-242); vector-search spike, OD-10 (SCRUM-191)

How it connects

Every handler my teammates wrote commits an audit entry with its change, and the audit log and history views read them back. The Foundry client is the one door to the model, and Mateus's similar items goes through it too.

Key files

```
services/ai/assetSearch.service.js
services/ai/foundry.client.js
pages/AuditLogPage.jsx
scripts/seedDemoCatalog.js
```

OD-10 measured 100 candidates against the live model: a query sharing no words with its target still ranked it first, so vector search waits until telemetry shows a need.

Code: The Model Only Ranks

Amber

code/backend/src/services/ai/assetSearch.service.js · runSearch()

```
const candidates = await pickCandidates(orgId, query);
const input = JSON.stringify({ query, assets: /* candidates */ });
let response;
try {
  response = await foundryRequest(orgId, { input }, { prompt: input });
} catch (err) {
  if (!(err instanceof ServiceUnavailableError))  throw err;
  return { result: await plainSearch(orgId, query) /* ... */ };
}
const answer = readAnswer(response);     // Zod contract
if (!answer) return { result: await plainSearch(orgId, query) };

const byId = new Map(candidates.map((a) => [String(a._id), a]));
const kept = answer.matches
```

1 2 3

code/backend/src/services/ai/foundry.client.js · foundryRequest()

```
let outcome = await attempt(url, body, deadline);
if (outcome.retryable) {              // network error, 429 or 5xx
  await pause(retryDelayMs(outcome.response), deadline);
  if (!deadline.aborted) outcome = await attempt(url, body, deadline);
}
```

4

Why it is safe

1. **The backend chooses**
   Up to 100 assets from this organization, retired ones excluded.
2. **Every failure is a fallback**
   Off, slow, down or off-contract: plain search, never an error.
3. **Unknown ids are dropped**
   An id the model invents is never looked up, so injection can only reorder.
4. **One retry, one deadline**
   Waits as long as Retry-After asks (max 3 s) inside the same 15 s budget.

Trimmed for the slide. Logs keep a prompt hash, latency and status, never the query or the answer (services/ai/README.md).

LIVE ON STAGING · ~1:45

Amber

Amber: Search and the Record, Running

1

Search by meaning

Member searches "something to record a lecture"

2

Restricted in results

Restricted badge on a result: Alexa's rule

3

AI search page

Admin: AI search page, status and fallbacks

4

Audit log

Audit log filtered to Dana: the whole demo

5

Chain of custody

Drone asset → History: chain of custody

Security and AI Use: The Final Account

Amber

Security requirements

SR-1, SR-2	Access control and tenant isolation	HELD
SR-3 – SR-7	Identity, sessions, transport	HELD
SR-9 – SR-14	Audit content, operations, scanning	HELD
SR-8	Audit immutability at the database layer	HELD

AI across the project

280

files with an AI-usage header

~92%

average stated AI contribution

84 to 0

"pending review" headers, Iteration 2 → now

- Worked: design and acceptance criteria first, then a failing test before every piece of AI code.
- Worked: reviewing AI code against those tests, and recording the reviewer in the header.
- Didn't: reviewing in bulk. Review is part of "done", not a cleanup task.

Fourteen SRs, each traced in SDD §6.10 to its design element and test. Header counts from main at cce52f5; full reflection in the project AI usage summary.

Iteration 2 Feedback: What We Did

Alex

Update the SDD for Iteration 2, with diagrams, and re-sign

SCRUM-127

Fix the progress-report Summary formulas

SCRUM-218

Record real decisions and named owners in minutes

Meetings 13 and 14 (SCRUM-220)

SPPP timeline: scope changes and actual hours

SCRUM-221

Update "Human Contributions" in AI-usage headers

84 pending →0 Pending (SCRUM-216)

One line per PR post: what it does, what to test

SCRUM-219

Coaching: a refactoring PR

SCRUM-225, PR #75

Coaching: demo videos out of the repo

Iter 0, 1 and 2 moved to Drive (PR #72 and 85);

The six "Before Iteration 3" items and the coaching notes from the Iteration 2 preliminary feedback (7 Oct). Status as of recording.

Alex

What we would keep, and what we would change

Keep

- Decide the design before the code, and number every open decision until it closes
- A failing test before the code that passes it
- One route table, one transition table, one audit rule
- Report what is not done instead of claiming it

Change

- Estimate from subtasks from day one (our first estimate was 70% low)
- Keep the board in step with main, not with memory
- Review AI-generated code as it lands, not in a batch

Repository github.com/BUMETCS673/CS673OLF26P3

Staging safe-drop-five.vercel.app

Thank you. Questions welcome in the discussion thread.